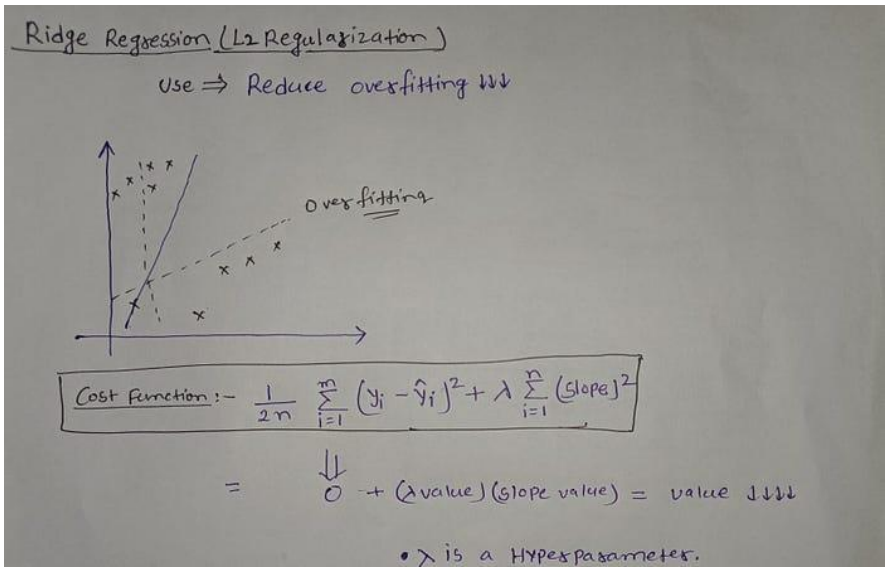
Introduction of Regularization

In the area of linear models, **ordinary least squares (OLS)** regression is one of the chief ways to estimate the relationship between variables. But OLS also has serious limitations: it tends to overfit, especially with multicollinearity and when the number of predictors is greater than observations. Regularization methods such as Ridge or Lasso regression, however, have been developed to tackle such problems.

1. Ridge Regression

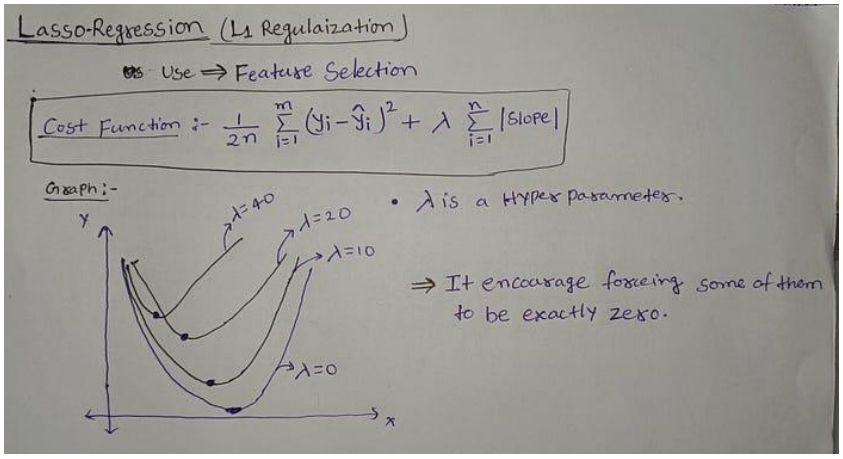
Ridge regression, also known as L2 regularization, utilizes a penalty term to improved ordinary least squares modeling. This penalty comprises the tuning parameter multiplied by the squared sum of the coefficient values.



The key intuition behind Ridge regression is that by penalizing large coefficients, it reduces the model's sensitivity to changes in the predictor variables and mitigates multicollinearity issues.

2. Lasso Regression

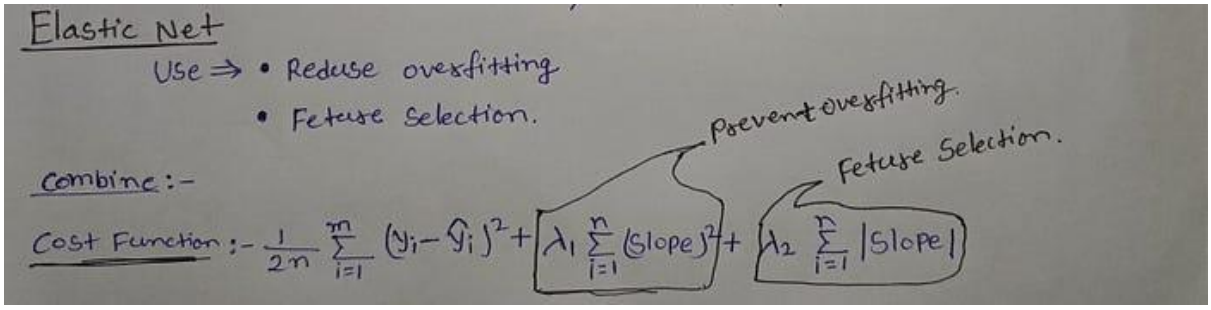
In Lasso regression, also known as L1 regularization, a penalty term is added to the coefficients that is proportional to their absolute values.



As Lasso regression puts some of these coefficient estimates exactly equal to zero it encourages sparsity. Because the coefficients of Lasso regression can be used to generate a sparse representation, Lasso regression is especially useful in feature selection: it automatically selects variables by directly setting some coefficients to zero.

3. Elastic Net Regression

Elastic Net regression combines the strengths of Ridge and Lasso regression, which involves both penalties $L1$ and $L2$. For Elastic Net regression, the objective function is:



Elastic Net regression is a versatile approach that effectively deals with both multicollinearity and feature selection and so is especially appropriate for datasets where both multicollinearity and sparse predictors feature simultaneously.

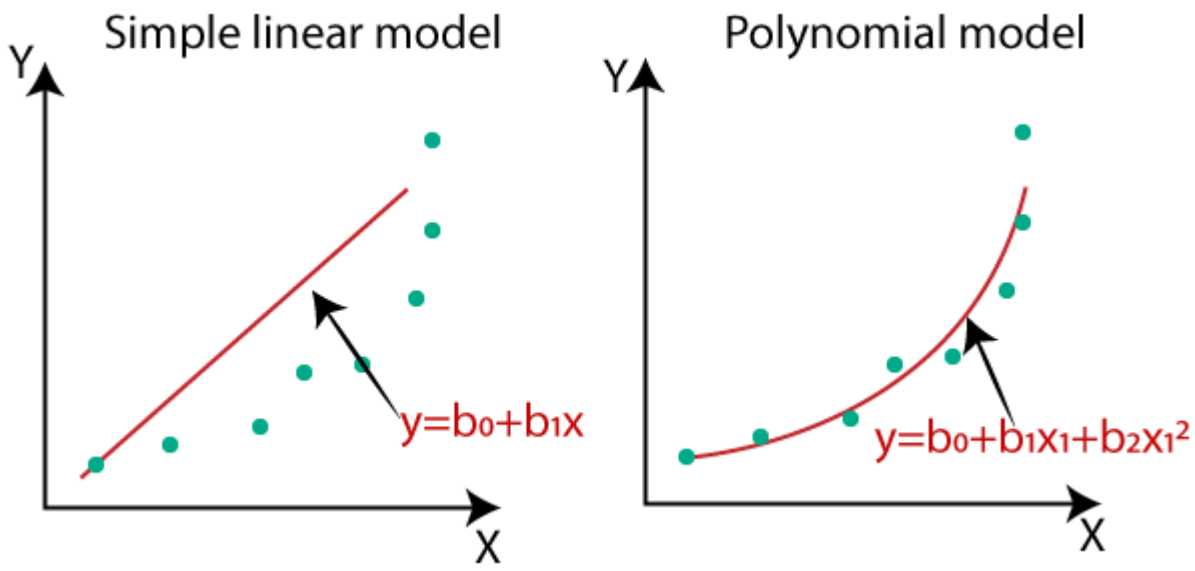
Comparing Ridge, Lasso Regression, and Elastic Net:

- Ridge regression can very effectively reduce the impact of multicollinearity and stabilize coefficient estimates, while it typically will not end up with exact zero coefficients that tallied-off each other in underdetermined cases.
- Lasso regression also actually includes a procedure for variable selection. It can set the tuned parameters down to zero thereof contents became more sparse in feature space.
- Elastic Net is a combination of Ridge and Lasso, giving us a generalization on the benefits both. It has been developed to take account of trade-offs between coefficient shrinkage and sparsity control.

What is Polynomial Regression?

- **Polynomial Regression** is a form of regression analysis in which the relationship between the independent variables and dependent variables are modeled in the nth degree polynomial.

We observe between the actual value and the best fit line,which we predicted and it seems that the actual value has some kind of curve in the graph and our line is no where near to cutting the mean of the points.



- **Polynomial Regression does not require the relationship between the independent and dependent variables to be linear in the data set,**This is also one of the main difference between the Linear and Polynomial Regression.
- Polynomial Regression is generally used when the points in the data are not captured by the Linear Regression Model and the Linear Regression fails in describing the best result clearly.

As we increase the degree in the model,it tends to increase the performance of the model.However,increasing the degrees of the model also increases the risk of over-fitting and under-fitting the data.

How to find the right degree of the equation?

In order to find the right degree for the model to prevent over-fitting or under-fitting, we can use:

1. **Forward Selection:**
   This method increases the degree until it is significant enough to define the best possible model.
2. **Backward Selection:**
   This method decreases the degree until it is significant enough to define the best possible model.

What Are Hyperparameters?

- Hyperparameters are parameters set before training and not learned from the data during training.
- Examples:
  - alpha in Ridge/Lasso regression (controls regularization strength)
  - n_neighbors in KNN

- max_depth in Decision Trees
- learning_rate in boosting algorithms

Why Tuning Hyperparameters?

- To improve model accuracy and generalization.
- To avoid:
  - Underfitting (too simple model)
  - Overfitting (too complex model)

Hyperparameter Tuning Methods

1. Grid Search

- Exhaustively searches through all possible combinations of provided parameter values.
- Slow, but accurate for small grids.

2. Randomized Search

- Randomly picks combinations for a fixed number of iterations.
- Faster and more efficient for large hyperparameter spaces.

Dataset Example

We'll use a simple synthetic dataset (10 rows) and tune alpha in Ridge Regression.

```
import numpy as np
from sklearn.linear_model import Ridge
from sklearn.model_selection import GridSearchCV, RandomizedSearchCV
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from scipy.stats import uniform
```

Step 1: Create a Tiny Dataset

```
# Generate 10 samples and 2 features
X, y = make_regression(n_samples=10, n_features=2, noise=5.0, random_state=1)
```

Grid Search with Ridge Regression

```
ridge = Ridge()
params = {'alpha': [0.01, 0.1, 1.0, 10.0, 100.0]
```

 # Set up Grid Search

```
grid = GridSearchCV(ridge, params, cv=3)
# Fit the model
grid.fit(X, y)
```

 # Output the best hyperparameters

```
print("Best Parameters from Grid Search:", grid.best_params_)
```

Explanation:

- Ridge(): model instance.
- params: dictionary with key alpha, values to test.
- GridSearchCV:
  - estimator = Ridge()
  - param_grid = params
  - cv = 3: 3-fold cross-validation.{ **Split the training data into 3 equal parts (folds)**, and perform training and validation **3 times**, each time using a different fold as validation and the remaining 2 as training.}
- grid.fit(X, y): trains Ridge model with all alpha values across folds.
- best_params_: returns the alpha that gave the best cross-validated score.

Randomized Search with Ridge Regression

```
param_dist = {'alpha': uniform(0.01, 10)}  # Range: [0.01, 10.01]
random = RandomizedSearchCV(Ridge(), param_distributions=param_dist, n_iter=5, cv=3, random_state=1)
random.fit(X, y)
```

 # Output the best hyperparameters

```
print("Best Parameters from Random Search:", random.best_params_)
```

Explanation:

- param_distributions: use scipy.stats.uniform to randomly sample alphas in range [0.01, 10.01].
- n_iter=5: test only 5 random alpha values.
- cv=3: 3-fold cross-validation.
- random_state=1: for reproducibility.

Summary

Method	Pros	Cons
Grid Search	Exhaustive & accurate	Time-consuming
Random Search	Faster, good for large spaces	May miss optimal point

Linear Classification Algorithms: KNN, logistic regression, classification metrics, ROC curve.

What is Logistic Regression?

- Logistic Regression is a Supervised statistical technique to find the probability of dependent variable(Classes present in the variable).
- Logistic regression uses functions called the ***logit functions***,*that helps* derive a relationship between the dependent variable and independent variables by predicting the probabilities or chances of occurrence.
- The logistic functions (also known as the *sigmoid functions*) convert the probabilities into binary values which could be further used for predictions.

Types of Logistic Regression:

1. **Binary Logistic Regression:**
   The dependent variable has only two 2 possible outcomes/classes.
   Example-Male or Female.
2. **Multinomial Logistic Regression:**
   The dependent variable has only two 3 or more possible outcomes/classes without ordering.
   Example: Predicting food quality.(Good,Great and Bad).
3. **Ordinal Logistic Regression:**
   The dependent variable has only two 3 or more possible outcomes/classes with ordering. Example: Star rating from 1 to 5

Prediction Formula

Given input features x and weights β, the model predicts probability y^ using the **sigmoid function**:

$$\hat{y} = \sigma(z) = \frac{1}{1+e^{-z}}, \quad \text{where } z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$

- $\hat{y} \in (0,1)$: predicted probability of class 1
- If $\hat{y} > 0.5$, predict class 1; else class 0.

Loss Function: Binary Cross-Entropy

To train the model, we minimize the **log loss** or **binary cross-entropy**:

$$\mathcal{L} = -\frac{1}{n}\sum_{i=1}^{n}\left[y^{(i)}\log(\hat{y}^{(i)}) + (1-y^{(i)})\log(1-\hat{y}^{(i)})\right]$$

Where:

- $y^{(i)} \in \{0,1\}$: true label
- $\hat{y}^{(i)}$: predicted probability
- n: number of training examples

This penalizes confident but wrong predictions heavily.

Cost Minimization

The parameters β\betaβ are learned by **minimizing the loss** using optimization algorithms like **Gradient Descent**:

$$\beta_j := \beta_j - \alpha \cdot \frac{\partial \mathcal{L}}{\partial \beta_j}$$

Where:

- α: learning rate

Metrics:

Confusion Matrix

The confusion matrix shows how predictions are distributed across actual classes. It's a 2×2 table for binary classification.

	Predicted: 0	**Predicted: 1**
Actual: 0 (TN)	True Negative	False Positive
Actual: 1 (TP)	False Negative	True Positive

TP: True Positive → predicted 1, actual 1 {True/False : Predicted Correct or not }

TN: True Negative → predicted 0, actual 0 {+/- : Actual Value}

FP: False Positive → predicted 1, actual 0

FN: False Negative → predicted 0, actual 1

Accuracy:

The ratio of correctly predicted observations to the total observations.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision:

Out of all the predicted positives, how many were actually positive (used to minimize false positives).

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall (Sensitivity)

Out of all actual positives, how many were correctly predicted (used to minimize false negatives).

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1-Score:

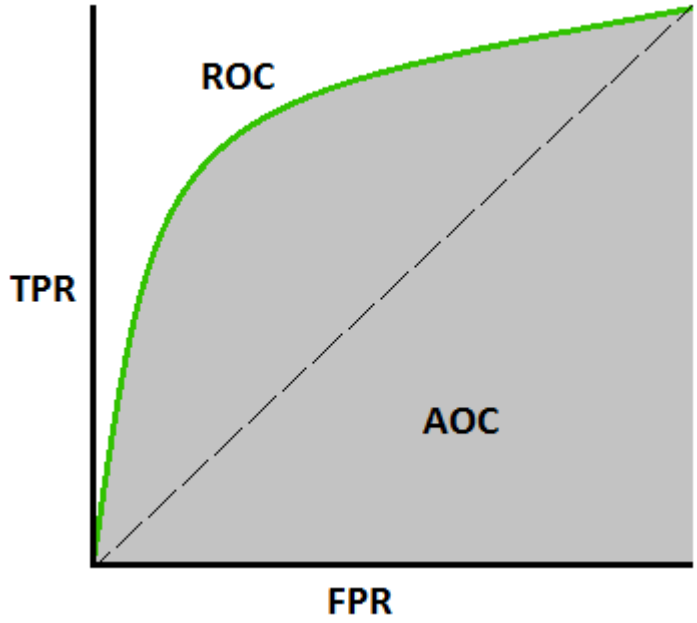
Harmonic mean of Precision and Recall — balances both.

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

What is the AUC - ROC Curve?

AUC - ROC curve is a performance measurement for the classification problems at various threshold settings. ROC is a probability curve and AUC represents the degree or measure of separability. It tells how much the model is capable of distinguishing between classes. Higher the AUC, the better the model is at predicting 0 classes as 0 and 1 classes as 1. By analogy, the Higher the AUC, the better the model is at distinguishing between patients with the disease and no disease.

The ROC curve is plotted with TPR against the FPR where TPR is on the y-axis and FPR is on the x-axis.



AUC - ROC Curve [Image 2] (Image courtesy: My Photoshopped Collection)

Defining terms used in AUC and ROC Curve.

TPR (True Positive Rate) / Recall /Sensitivity

$$\text{TPR /Recall / Sensitivity} = \frac{TP}{TP + FN}$$

Specificity

$$\text{Specificity} = \frac{TN}{TN + FP}$$

FPR

$$\text{FPR} = 1 - \text{Specificity}$$

$$= \frac{FP}{TN + FP}$$

How to speculate about the performance of the model?

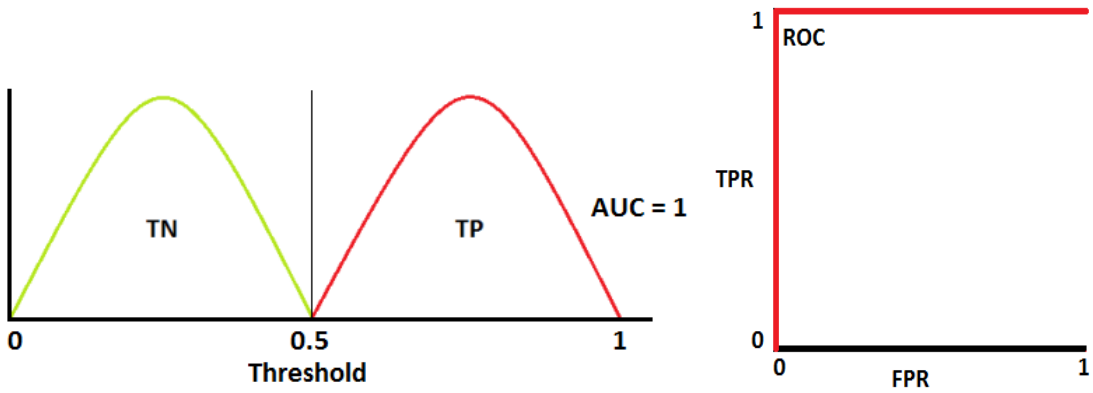
An excellent model has AUC near to the 1 which means it has a good measure of separability. A poor model has an AUC near 0 which means it has the worst measure of

separability. In fact, it means it is reciprocating the result. It is predicting 0s as 1s and 1s as 0s. And when AUC is 0.5, it means the model has no class separation capacity whatsoever.

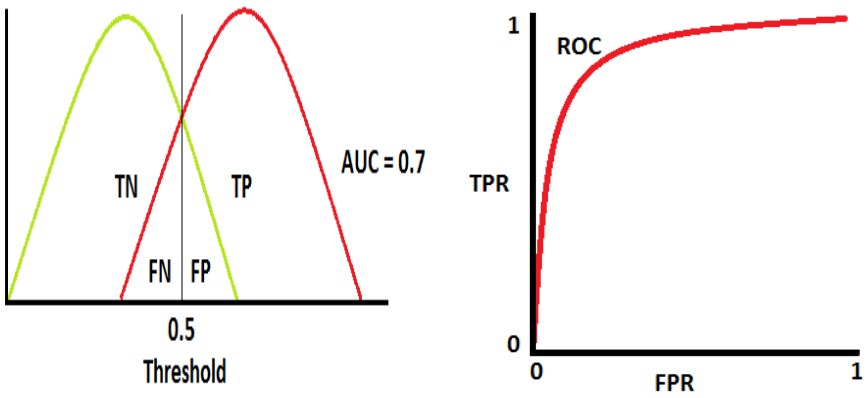
Let’s interpret the above statements.

As we know, ROC is a curve of probability. So let's plot the distributions of those probabilities:

Note: Red distribution curve is of the positive class (patients with disease) and the green distribution curve is of the negative class(patients with no disease).



This is an ideal situation. When two curves don’t overlap at all means model has an ideal measure of separability. It is perfectly able to distinguish between positive class and negative class.



When two distributions overlap, we introduce type 1 and type 2 errors. Depending upon the threshold, we can minimize or maximize them. When AUC is 0.7, it means there is a 70% chance that the model will be able to distinguish between positive class and negative class.